

Lab 8 — Manage Azure Resources with Cloud Shell and Azure CLI

Hands-on training lab · Junior cloud support portfolio

Resource group	Region	Shell	Storage accounts
rg-gp-cli-demo	North Europe	Cloud Shell (Bash)	2

Objective

The lab focused on using Azure Cloud Shell and the Azure CLI to create, inspect, tag, query, and remove Azure resources from the command line instead of the Azure Portal.

Technologies

Microsoft Azure · Azure Cloud Shell · Azure CLI · Azure Resource Manager · Azure Storage · JMESPath

What I did (shown in the screenshots)

- Opened Cloud Shell and used Bash
- Verified the Azure subscription with `az account show` and `az account list`
- Looked at the Azure CLI help output
- Created the resource group `rg-gp-cli-demo` in North Europe with the CLI
- Worked with two storage accounts in that group: `stgpcldemo0165606495` and `stgpcldemo0265606495`
- Listed and filtered resources with `az resource list`
- Inspected a storage account and used `--query` (JMESPath) to pull out single values
- Checked the resource group in the Azure Portal
- Deleted the resource group from the CLI and confirmed it was gone

Tags: tagging was part of the lab, but I did not capture screenshots of the tagging commands or results. Page 4 says exactly what is and is not shown.

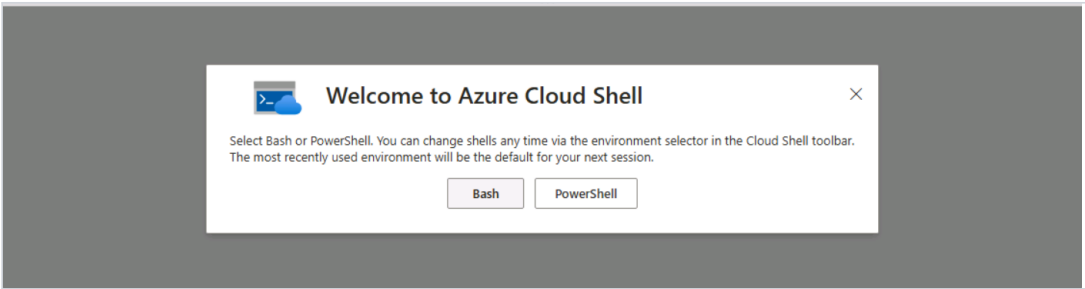


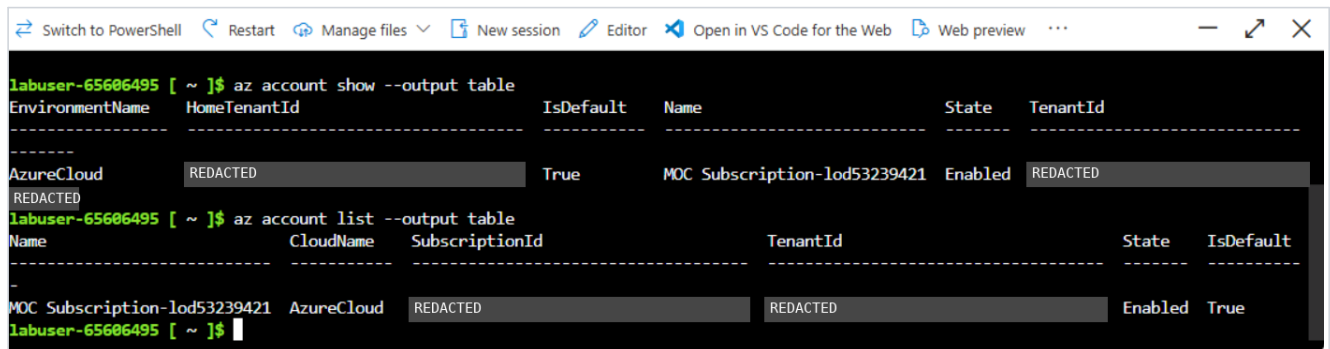
Figure 1 — The Cloud Shell welcome dialog offering Bash or PowerShell. I chose Bash.

Cloud Shell and Azure CLI

After opening Cloud Shell in Bash (Figure 1, page 1), the first job was to confirm which account and subscription the CLI was using, then look around the CLI.

Verifying the account and subscription

`az account show --output table` and `az account list --output table` both returned one subscription, in **Enabled** state and marked as the default. The subscription and tenant IDs are redacted in the screenshot.

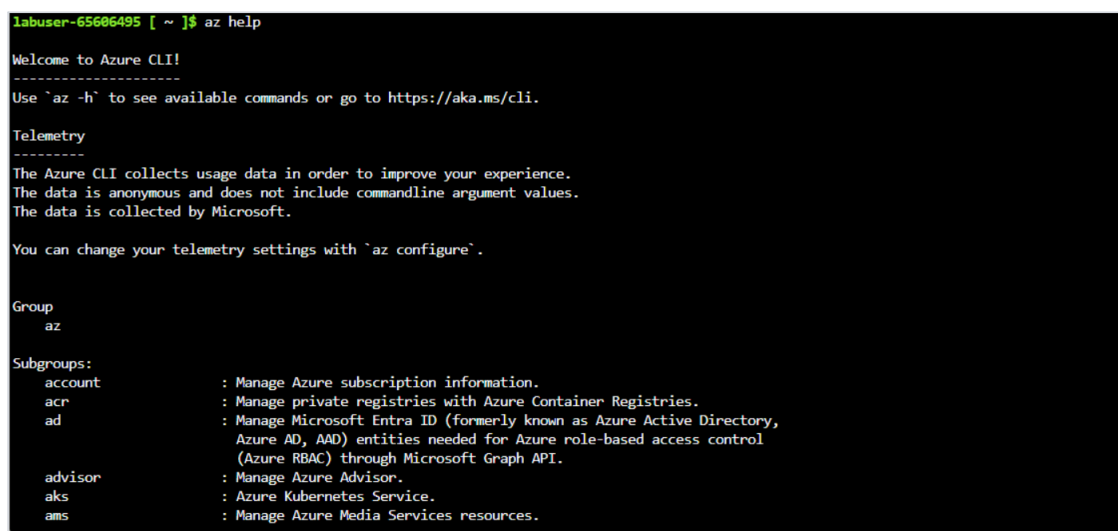
The screenshot shows a Cloud Shell terminal window with a dark background. At the top, there's a toolbar with icons for switching to PowerShell, restarting, managing files, creating a new session, editing, opening in VS Code, and web preview. The terminal shows two commands being executed. The first command is 'az account show --output table', which returns a table with columns: EnvironmentName, HomeTenantId, IsDefault, Name, State, and TenantId. The second command is 'az account list --output table', which returns a table with columns: Name, CloudName, SubscriptionId, TenantId, State, and IsDefault. In both tables, the TenantId and SubscriptionId fields are redacted with grey boxes.

```
labuser-65606495 [ ~ ]$ az account show --output table
EnvironmentName  HomeTenantId  IsDefault  Name  State  TenantId
-----
AzureCloud      REDACTED      True       MOC Subscription-lod53239421  Enabled  REDACTED
labuser-65606495 [ ~ ]$ az account list --output table
Name  CloudName  SubscriptionId  TenantId  State  IsDefault
-----
MOC Subscription-lod53239421  AzureCloud  REDACTED  REDACTED  Enabled  True
labuser-65606495 [ ~ ]$
```

Figure 2 — Account and subscription check. Subscription ID and tenant IDs redacted.

Exploring the CLI help

`az help` shows the welcome text and the list of command groups, such as `account`, `acr`, `ad`, `advisor`, `aks` and `ams`. Each group has its own subcommands and help.

The screenshot shows a Cloud Shell terminal window with a dark background. The terminal displays the output of the 'az help' command. It starts with a welcome message, followed by instructions on how to see available commands. Then, it shows telemetry information. Finally, it lists the command groups and their subgroups with brief descriptions.

```
labuser-65606495 [ ~ ]$ az help
Welcome to Azure CLI!
-----
Use `az -h` to see available commands or go to https://aka.ms/cli.

Telemetry
-----
The Azure CLI collects usage data in order to improve your experience.
The data is anonymous and does not include commandline argument values.
The data is collected by Microsoft.

You can change your telemetry settings with `az configure`.

Group
  az

Subgroups:
  account      : Manage Azure subscription information.
  acr          : Manage private registries with Azure Container Registries.
  ad           : Manage Microsoft Entra ID (formerly known as Azure Active Directory,
                Azure AD, AAD) entities needed for Azure role-based access control
                (Azure RBAC) through Microsoft Graph API.
  advisor      : Manage Azure Advisor.
  aks          : Azure Kubernetes Service.
  ams          : Manage Azure Media Services resources.
```

Figure 3 — Output of `az help` (top of the command group list).

Not captured: I do not have screenshots of `az group --help` or `az group create --help`, so they are not shown here. The end of the region list output is visible at the top of Figure 4 on the next page.

Creating and Managing Resources

Resource group

The end of the region list output (Figure 4, top) shows the location name format, for example `norwaywest` and `ukwest`. I used `northeurope` as the location. My first attempt, `az group --name rg-gp-cli-demo --location northeurope`, failed because I left out the `create` subcommand, and the CLI reported the name as not recognized. Re-running it with `az group create` worked and returned **provisioningState: Succeeded**. `az group show` then confirmed the group exists.

```
Norway West          norwaywest          (Europe) Norway West
Switzerland West     switzerlandwest     (Europe) Switzerland West
UAE Central          uaecentral          (Middle East) UAE Central
Brazil Southeast      brazilsoutheast     (South America) Brazil Southeast
UK West              ukwest             (UK) UK West
labuser-65606495 [ ~ ]$ az group --name rg-gp-cli-demo --location northeurope
'rg-gp-cli-demo' is misspelled or not recognized by the system.

https://aka.ms/cli_ref
Read more about the command in reference docs
labuser-65606495 [ ~ ]$ az group create --name rg-gp-cli-demo --location northeurope
{
  "id": "/subscriptions/REDACTED/resourceGroups/rg-gp-cli-demo",
  "location": "northeurope",
  "managedBy": null,
  "name": "rg-gp-cli-demo",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
labuser-65606495 [ ~ ]$ az group show --name rg-gp-cli-demo --output table
Location      Name
-----
northeurope   rg-gp-cli-demo
```

Figure 4 — End of the region list, the failed command without `create`, the successful `az group create`, and `az group show`. Subscription ID redacted.

Storage accounts, listing and filtering

The resource group holds two storage accounts: `stgpcldemo0165606495` (`northeurope`) and `stgpcldemo0265606495` (`eastus`). Both show a status of `Succeeded`. I listed them with `az resource list`, then repeated the list with `--resource-type Microsoft.Storage/storageAccounts`. Both commands return the same two accounts because they are the only resources in the group.

I then ran `az storage account show` on the first account. The table shows access tier **Hot**, blob public access **False**, HTTPS-only **True**, provisioning state **Succeeded** and status **available**.

```
labuser-65606495 [ ~ ]$ az resource list --resource-group rg-gp-cli-demo --output table
Name                               ResourceGroup Location Type                               Status
-----
stgpcldemo0165606495               rg-gp-cli-demo northeurope Microsoft.Storage/storageAccounts Succeeded
stgpcldemo0265606495               rg-gp-cli-demo eastus Microsoft.Storage/storageAccounts Succeeded
labuser-65606495 [ ~ ]$ az resource list \
> --resource-group rg-gp-cli-demo \
> --resource-type Microsoft.Storage/storageAccounts \
> --output table
Name                               ResourceGroup Location Type                               Status
-----
stgpcldemo0165606495               rg-gp-cli-demo northeurope Microsoft.Storage/storageAccounts Succeeded
stgpcldemo0265606495               rg-gp-cli-demo eastus Microsoft.Storage/storageAccounts Succeeded
labuser-65606495 [ ~ ]$ az storage account show \
> --name stgpcldemo0165606495 \
> --resource-group rg-gp-cli-demo \
> --output table
AccessTier  AllowBlobPublicAccess  AllowCrossTenantReplication  CreationTime  EnableHttpsTrafficOnly  Kind
-----
Hot         False                  False                        2026-09-28T05:33:50.250090+00:00  True                    StorageV
2 northeurope TLS1_0 stgpcldemo0165606495 northeurope Succeeded rg-gp-cli-demo available
```

Figure 5 — `az resource list`, the same list filtered by `--resource-type`, and `az storage account show` for `stgpcldemo0165606495`.

Configuration correction: I created the first storage account with `Standard_RAGRS` by mistake and changed it to `Standard_LRS`. That change is not in a screenshot. The `Standard_LRS` result is shown on page 4. I did not capture the storage account creation commands either.

Tags and JMESPath Queries

Using --query

Azure CLI can filter its output with the `--query` argument, which uses the JMESPath query language. Instead of reading through a full JSON result, I can ask for one field and combine it with `--output table`. Two examples from the lab:

```
labuser-65606495 [ ~ ]$ az storage account show \
--name stgpclidemo0165606495 \
--resource-group rg-gp-cli-demo \
--query sku.name \
--output table
Result
-----
Standard_LRS
labuser-65606495 [ ~ ]$
```

Figure 6 — `--query sku.name` on `stgpclidemo0165606495` returns `Standard_LRS`.

```
labuser-65606495 [ ~ ]$ az storage account show \
--name stgpclidemo0265606495 \
--resource-group rg-gp-cli-demo \
--query provisioningState \
--output table
Result
-----
Succeeded
labuser-65606495 [ ~ ]$
```

Figure 7 — `--query provisioningState` on `stgpclidemo0265606495` returns `Succeeded`.

Tags

The lab included tagging the resource group with `environment=test` and `department=it-ops`, tagging the individual resources, and querying them. I did not capture screenshots of the tagging commands, the tag queries, or any filtering or counting queries. The JSON output I did capture shows an empty tags value, so I am not showing tag results in this document.

This page only shows what the screenshots prove: two single-value queries. It does not claim tag or counting results.

Portal Comparison and Cleanup

Comparing with the Azure Portal

The Portal's Resource groups page lists `rg-gp-cli-demo` in North Europe, in the same subscription, next to `cloud-shell-storage-northeurope`. This matches the resource group I created in the CLI. The screenshot does not show tags or the contents of the group, so I am not claiming a tag check in the Portal.

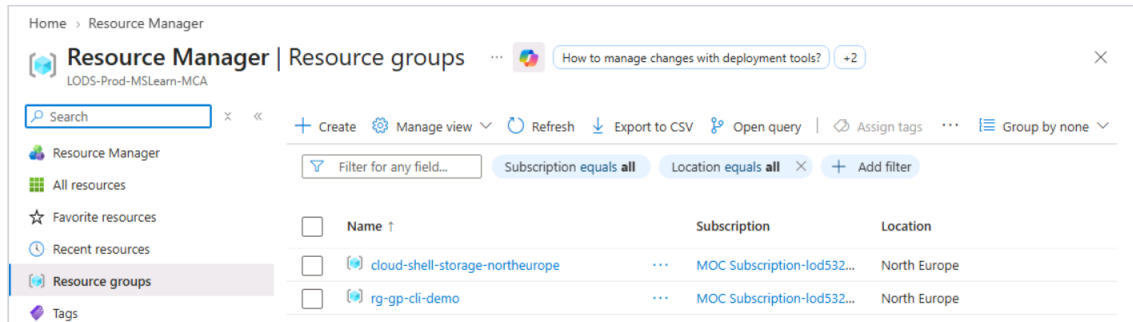


Figure 8 — Azure Portal, Resource groups list showing `rg-gp-cli-demo`.

Deleting the resource group

I deleted the group with `az group delete --name rg-gp-cli-demo --yes --no-wait`. The `--no-wait` option returns straight away without waiting for the deletion to finish, and `az group show` still found the group right after. Running the delete and show commands again returned **ResourceGroupNotFound**, which confirmed the group was removed.

```
labuser-65606495 [ ~ ]$ az group delete --name rg-gp-cli-demo --yes --no-wait
labuser-65606495 [ ~ ]$ az group show --name rg-gp-cli-demo --output table
Location      Name
-----
northeurope   rg-gp-cli-demo
labuser-65606495 [ ~ ]$ az group show --name rg-gp-cli-demo --output table
Location      Name
-----
northeurope   rg-gp-cli-demo
labuser-65606495 [ ~ ]$ az group delete --name rg-gp-cli-demo --yes --no-wait
(ResourceGroupNotFound) Resource group 'rg-gp-cli-demo' could not be found.
Code: ResourceGroupNotFound
Message: Resource group 'rg-gp-cli-demo' could not be found.
labuser-65606495 [ ~ ]$ az group show --name rg-gp-cli-demo --output table
(ResourceGroupNotFound) Resource group 'rg-gp-cli-demo' could not be found.
Code: ResourceGroupNotFound
Message: Resource group 'rg-gp-cli-demo' could not be found.
labuser-65606495 [ ~ ]$
```

Figure 9 — Delete command, the group still found straight after, then **ResourceGroupNotFound** for both delete and show.

Final cleanup

The lab listed the Cloud Shell storage as an optional cleanup item. `cloud-shell-storage-northeurope` appears in the Portal screenshot taken before cleanup, and no screenshot shows it being deleted, so I am not claiming that it was.

What I Learned

- How to do basic Azure resource management from Cloud Shell with the CLI instead of relying only on the Portal.
- How the command groups fit together: `az account`, `az group`, `az resource` and `az storage account`.
- How to create and inspect a resource group and storage accounts from the command line.
- How to use `--output table` and `--query (JMESPath)` to get short, readable results.
- How to check the same resource group in both the CLI and the Portal.
- That a missing subcommand gives an error, and how to read it and fix the command.
- How to clean up a lab by deleting the resource group and confirming it is gone.

Notes on the evidence

- Subscription and tenant IDs are redacted in Figures 2 and 4. No passwords, access keys, SAS tokens, connection strings or personal email addresses appear in the screenshots.
- This was a training lab, not a real deployment.
- Not shown by screenshots: `az group --help` and `az group create --help`, the full region list, storage account creation commands, tagging, tag queries, and the Cloud Shell storage check.